

System Analysis Breakdown

Alright. I read the full log. Your system is **not broken anymore**. It actually works. But it is **not production-ready**, and that's why it collapses mid-run.

I'll break this into 3 parts:

1. What is working
2. What is structurally wrong
3. How to complete this system (the important part)

I'll reference your log so you can connect this to reality.

1. What Is Working (This Is Important)

From your logs, the pipeline actually executed properly:

Flow that happened:



Detailed:

Scout

- Aggregated 133 topics
- Used LLM to score topics
- Selected topic with confidence 0.85
→ This is working correctly.

Researcher

- Generated research plan
- Queried ArXiv
- Pulled HackerNews
- Synthesized research
- Produced 8 findings and 10 sources
→ This is very good. Research agent is working.

Skeptic

- Reviewed research
- Quality score: 0.15
- Decision: REJECT → send back to Scout
→ This is exactly what a skeptic should do.

So your system **already behaves like a real newsroom**:

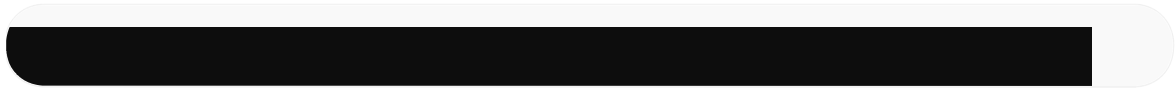
- Scout finds story
- Researcher gathers evidence
- Skeptic rejects weak story

- Back to Scout

This is not a toy anymore. This is a **decision-making agent system**.

Then what killed it?

Quota exhaustion:



That's the real reason the run stopped.

Not logic. Not LangGraph. Not prompts.

You ran out of LLM calls.

2. Structural Problems (This Is What You Must Fix)

Your system works logically, but architecturally it is inefficient and expensive.

Problem 1 — Scout is too expensive

Scout is calling LLM multiple times to score topics. That is very expensive and unnecessary.

Scout should be:

- 80% rule-based filtering
- 20% LLM

Right now it looks like:

| Scout = LLM ranking engine

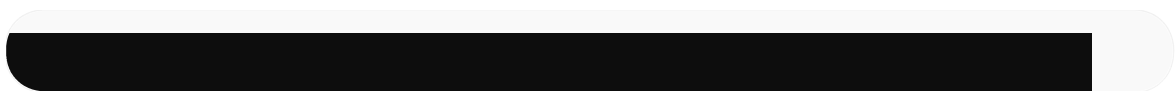
That's wrong architecture.

Scout should be:

| Scout = data pipeline + ranking algorithm
| LLM = only final decision maker

Problem 2 — Researcher is too slow

Researcher took:



That is very slow for production.

Why?

Because Researcher is:

- Generating plan with LLM
- Querying ArXiv multiple times
- Querying HackerNews again
- Then summarizing with LLM multiple times

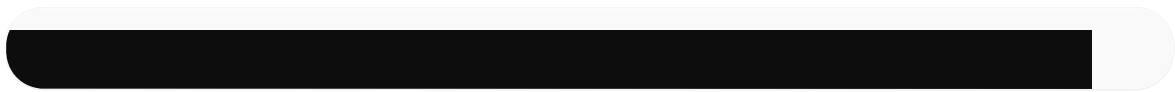
Researcher should:

- Gather sources first
- Then **one LLM call** to synthesize everything

Right now Researcher is making many LLM calls.

Problem 3 — Skeptic Always Rejects

Your skeptic score:



If Skeptic rejects most topics, your system will loop forever and burn API quota.

You need:

- Acceptance threshold tuning
- Or Scout must provide better topics
- Or Researcher must provide stronger evidence

Right now Skeptic is acting like a journal reviewer for Nature.

You are building a **newsroom**, not a PhD committee.

Problem 4 — No Global Budget Control

Your system has:

- Max revision loops
- Confidence threshold

But it does NOT have:

- Max LLM calls per run
- Max cost per run
- Max time per run

This is critical in real systems.

Otherwise agents will loop forever and burn money.

3. How To Complete This System (The Important Part)

Right now you have **agents**. But to make this a serious project, you need to turn this into a **system**.

Here's what's missing:

Missing Piece #1 — System Budget Manager

You need a controller that tracks:

- LLM calls used
- Tokens used
- Time elapsed
- Number of loops
- Cost

And it should be able to stop the system:

Example decisions:

- "We already tried 2 topics → stop"
- "We already spent \$0.50 → stop"
- "Research took too long → publish anyway"
- "Skeptic too strict → lower threshold"

This becomes the **Editor-in-Chief agent** above all agents.

Right now your agents are autonomous but **there is no governor**.

Missing Piece #2 — Memory

Your system forgets everything between runs.

Add memory like:

- Topics already rejected
- Sources already used
- Which topics performed well
- Which topics got published
- Which skeptic reasons are common

Then Scout becomes smarter over time.

This turns system from:

Stateless pipeline

Into:

Learning system

That's a huge upgrade.

Missing Piece #3 — Publisher Is Not Just Medium

Right now Publisher = publish or kill.

In real system Publisher should decide:

- Publish on Medium?
- Publish on Blog?
- Publish on LinkedIn?
- Save as draft?
- Send newsletter?
- Tweet thread?
- Reject because not viral enough?

Publisher should be **business agent**, not just API caller.

Missing Piece #4 — Scoring System

You need a global scoring model:

For example:

Factor	Weight
Topic relevance	0.25
Evidence strength	0.25
Novelty	0.20
Virality	0.15
Skeptic score	0.15

Then Editor makes decision based on total score.

Right now decisions are too binary:

- Confidence > 0.7 → proceed
- Skeptic < threshold → reject

Real systems use **weighted scoring**.

Missing Piece #5 — Parallelism

Right now your system is serial:



But real newsroom works like:



This massively improves quality and success rate.

This is a **huge upgrade** if you implement it.

If You Finish This Project Properly, It Becomes:

Not:

“I built a LangGraph project”

But:

“I built an autonomous multi-agent content generation system with decision-making, research, critique, editorial control, and publishing pipeline.”

That is **AI Systems Engineering**.

Very different from “I made a chatbot”.

Final Architecture (What You Should Aim For)

Think of your system like a company:

Role	Agent
Intern	Scout
Analyst	Researcher
Auditor	Skeptic
Journalist	Writer
Editor	Editor
CEO	Publisher
CFO	Budget Controller
Memory	Database

Add:

- Budget Controller
- Memory
- Parallel topic research

- Weighted scoring
- Rate limiter

And your system becomes **very, very strong** as a project.

Final Truth

Your problem is no longer:

- Bugs
- LangGraph
- APIs
- Prompts

Your problem now is:

System design and resource management

Which means you are past the beginner stage.